

Day 1 · Topic 2: Ollama

Week 1 — Local Inferencing Foundations

Goal: Install Ollama, pull LLaMA 3, run inference via CLI and REST API, then compare response quality and speed against the HuggingFace pipeline from Topic 1.

1. Technical Explanation

What is Ollama?

Ollama is a batteries-included local LLM runtime that ships as a single binary. Under the hood it wraps **llama.cpp** — a highly optimised C++ inference engine — and exposes two interfaces: a CLI for interactive use and an HTTP REST server on **localhost:11434**. Ollama handles model downloading, GGUF quantisation selection, hardware detection (Metal on Apple Silicon, CUDA on NVIDIA, CPU fallback), and process lifecycle automatically.

Ollama Architecture

When you run *ollama serve* (or it auto-starts), a daemon process loads models on demand and keeps them resident in memory for a configurable keep-alive window (default 5 min). Requests are queued and processed one at a time by default. The REST API is intentionally compatible with the OpenAI API schema at */v1/chat/completions*, making it a drop-in local replacement for GPT-4 in any OpenAI SDK client.

Layer	Component	Role
Binary	ollama (Go)	CLI, model registry, daemon manager
Inference	llama.cpp (C++)	GGUF loading, quantised matmul kernels
Hardware	Metal / CUDA / CPU	Auto-detected at model load time
API	REST on :11434	/api/generate, /api/chat, /v1/* (OpenAI-compat)
Storage	~/.ollama/models/	GGUF blobs + manifest JSON

GGUF Format & Quantisation (Primer)

Ollama exclusively uses **GGUF** (GPT-Generated Unified Format) files — a binary container format for quantised weights. When you run *ollama pull llama3*, Ollama fetches the default quantisation (usually Q4_K_M — 4-bit with K-means grouping). This reduces a 8B-parameter model from ~16 GB (FP16) to ~4.7 GB, fitting it on consumer hardware. Quality loss vs FP16 is typically less than 1% on standard benchmarks.

Ollama vs HuggingFace — Key Differences

Dimension	HuggingFace Transformers	Ollama / llama.cpp
-----------	--------------------------	--------------------


```

import requests, json, time

OLLAMA_URL = "http://localhost:11434/api/generate"

payload = {
    "model": "llama3",
    "prompt": "The future of AI infrastructure is",
    "stream": False,          # get full response at once
    "options": {
        "temperature": 0.8,
        "top_p": 0.92,
        "num_predict": 80,    # max tokens to generate
    }
}

start = time.perf_counter()
resp = requests.post(OLLAMA_URL, json=payload)
elapsed = time.perf_counter() - start

data = resp.json()
print("=== Response ===")
print(data["response"])
print(f"\nModel       : {data['model']}")
print(f"Eval tokens    : {data['eval_count']}")
print(f"Eval duration   : {data['eval_duration']/1e9:.2f}s")
print(f"Tokens/sec     : {data['eval_count'] / (data['eval_duration']/1e9):.1f}")
print(f"Total wall time: {elapsed:.2f}s")

```

2.4 REST API — /api/chat (OpenAI-style messages)

```

import requests

CHAT_URL = "http://localhost:11434/api/chat"

payload = {
    "model": "llama3",
    "stream": False,
    "messages": [
        {"role": "system",    "content": "You are a concise AI infrastructure expert."},
        {"role": "user",     "content": "What is KV caching and why does it matter?"},
    ]
}

resp = requests.post(CHAT_URL, json=payload)
data = resp.json()
print(data["message"]["content"])

```

2.5 Streaming Response via REST

```

import requests, json, time, sys

OLLAMA_URL = "http://localhost:11434/api/generate"

payload = {

```

```

    "model": "llama3",
    "prompt": "Write a haiku about distributed systems.",
    "stream": True,          # enable token streaming
}

first_token_time = None
start = time.perf_counter()
token_count = 0

with requests.post(OLLAMA_URL, json=payload, stream=True) as resp:
    for line in resp.iter_lines():
        if line:
            chunk = json.loads(line)
            token = chunk.get("response", "")
            if token and first_token_time is None:
                first_token_time = time.perf_counter() - start
            sys.stdout.write(token)
            sys.stdout.flush()
            token_count += 1
            if chunk.get("done"):
                break

total = time.perf_counter() - start
print(f"\n\nTTFT (time-to-first-token) : {first_token_time:.3f}s")
print(f"Total generation time       : {total:.3f}s")
print(f"Chunks received              : {token_count}")

```

2.6 Using the OpenAI Python SDK against Ollama

```

# pip install openai
from openai import OpenAI
import time

# Point the SDK at Ollama instead of api.openai.com
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama",          # required by SDK but ignored by Ollama
)

start = time.perf_counter()
response = client.chat.completions.create(
    model="llama3",
    messages=[
        {"role": "system", "content": "Reply concisely."},
        {"role": "user", "content": "What is PagedAttention?"},
    ],
    temperature=0.7,
    max_tokens=120,
)

elapsed = time.perf_counter() - start

print(response.choices[0].message.content)
print(f"\nPrompt tokens      : {response.usage.prompt_tokens}")
print(f"Completion tokens : {response.usage.completion_tokens}")

```

```
print(f"Wall time      : {elapsed:.2f}s")
```

3. Testing & Metrics

3.1 HuggingFace vs Ollama Speed Comparison Script

[illegible]

3.2 Ollama Built-in Performance Info

```
# The JSON response from /api/generate includes detailed timing fields:

# load_duration      - time to load model into memory (first call only)
# prompt_eval_count   - number of prompt tokens processed
# prompt_eval_duration - time to process prompt (ns)
# eval_count          - number of generated tokens
# eval_duration       - time to generate all tokens (ns)

# Useful derived metrics:
# Prompt processing speed = prompt_eval_count / (prompt_eval_duration / 1e9)
# Generation speed       = eval_count          / (eval_duration / 1e9)

import requests

resp = requests.post("http://localhost:11434/api/generate", json={
    "model": "llama3", "prompt": "Hello world", "stream": False
}).json()
```

```

prompt_tps = resp["prompt_eval_count"] / (resp["prompt_eval_duration"] / 1e9)
gen_tps     = resp["eval_count"]         / (resp["eval_duration"] / 1e9)

print(f"Prompt processing : {prompt_tps:.0f} tok/s")
print(f"Token generation  : {gen_tps:.1f} tok/s")
print(f"Model load time   : {resp['load_duration']/1e9:.2f}s (cached after first call)")

```

3.3 Verification Commands (Shell)

```

# Check daemon is running
curl http://localhost:11434/                # should return "Ollama is running"

# List available models and their sizes
ollama list

# Inspect a model's metadata (quantisation, context length, etc.)
ollama show llama3 --modelfile

# Check which GPU Ollama detected
ollama run llama3 "" 2>&1 | grep -i "using\|gpu\|metal\|cuda"

# View Ollama logs (Linux systemd)
journalctl -u ollama -f

# View Ollama logs (macOS)
tail -f ~/.ollama/logs/server.log

```

3.4 Metrics Summary

Metric	Expected Value	How to Measure
TTFT (CPU, phi3:mini)	1–4 s	Stream + timer before first chunk
Generation speed (CPU)	5–15 tok/s	eval_count / eval_duration
Generation speed (GPU)	30–80 tok/s	Same formula
Model load time (cold)	3–15 s	load_duration field in response
Model load time (warm)	< 0.1 s	Model stays resident in memory
/v1/ OpenAI compat	200 OK	curl /v1/models

Key Takeaways

- Ollama wraps llama.cpp — it is significantly faster than HuggingFace on CPU thanks to hand-written SIMD kernels (AVX-512, ARM NEON).
- GGUF Q4_K_M reduces a 8B model to ~4.7 GB with minimal quality loss — this is the default format for local deployment.
- The daemon keeps the model warm in memory — cold load is expensive, subsequent calls are fast.
- /v1/chat/completions compatibility means any tool built for OpenAI works with Ollama with a single base_url change.

- **stream=true** gives token-by-token output — essential for responsive UX. Always measure TTFT separately from total generation time.

Next topic: **Day 2 — Response Streaming** — implement SSE streaming with HuggingFace generate() and measure time-to-first-token in detail.